Principles of Programming Languages
Functional Programming

Andrei Arusoaie[1]

[1]Department of Computer Science

Outline

Outline

Outline

Paradigms

- Imperative Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓
- Declarative Programming
  - Functional Programming
  - Logic Programming

Outline

General facts about PLs - I

General facts about PLs - I

- Computational model based on:

  state + *modifiable variable* + *assignment*

General facts about PLs - I

- Computational model based on:

  state + *modifiable variable* + *assignment*

- *Computation proceeds by modifying values stored in locations*

General facts about PLs - I

- Computational model based on:

  state + *modifiable variable* + *assignment*

- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*

General facts about PLs - I

- Computational model based on:

  state + *modifiable variable* + *assignment*

- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- Conventional PL are based on this computational model

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?
  - ... by *rewriting* expressions

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?
  - ... by *rewriting* expressions
  - The computation is expressed in terms of modifications of the environment

General facts about PLs - II

- However, this is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?
  - ... by *rewriting* expressions
  - The computation is expressed in terms of modifications of the environment
  - “Pure” principles: do not modify the ‘state’; no side-effects

Functional programming: history + characteristics

- This paradigm is as old as the imperative one

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine vs. λ-calculus (1936)
  - Church-Turing thesis

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine vs. λ-calculus (1936)
  - Church-Turing thesis
- Ingredients: higher-order functions + recursion

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine vs. λ-calculus (1936)
  - Church-Turing thesis
- Ingredients: higher-order functions + recursion
- PLs: Haskell, Miranda (pure) + Lisp, ML, Scheme, ...

Some mathematical background

- Math function: $f(x) = x^2$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
  - $f(x)$ is the expression that results after the transformation happens

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
  - $f(x)$ is the expression that results after the transformation happens
- Functions = *expressible* values

More about writing function application

- Classical: $f(2)$

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:
  - Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:
  - Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax
  - Application: $(\backslash x \rightarrow x * x)\ 2$

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:
  - Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax
  - Application: $(\backslash x \rightarrow x * x)\ 2$
  - Higher-order functions, currying, ...

(Untyped) λ - calculus

λ-terms are *variables*, *abstractions*, or *applications*:

$$
\begin{array}{lcll}
t & ::= & x & \text{// where } x \in X \\
 & | & \lambda x.t & \text{// where } x \in X \text{ and } t \text{ is a } \lambda\text{-term} \\
 & | & t\ s & \text{// where } t \text{ and } s \text{ are } \lambda\text{-terms}
\end{array}
$$

Abstractions

The construction $\lambda x.t$ is called *abstraction*.

If in mathematics we define a function as below

$$f(x) = x^2 + 2$$

then in λ-calculus this function can be encoded as an abstraction:

$$\lambda x.x^2 + 2.$$

Abstractions and α-equivalence

The function $f(x) = x^2 + 2$ takes x as input and computes the square of x and adds 2.

The abstraction $\lambda x.x^2 + 2$ does the same thing, except that we do not indicate a name for the function.

Abstractions and α-equivalence

The function $f(x) = x^2 + 2$ takes x as input and computes the square of x and adds 2.

The abstraction $\lambda x.x^2 + 2$ does the same thing, except that we do not indicate a name for the function.

The abstraction operator in the term $\lambda x.t$ *binds* x, that is, in t, x is seen as an input. In $\lambda x.x^2 + 2$, the input is x and t is $x^2 + 2$.

Abstractions and α-equivalence

The function $f(x) = x^2 + 2$ takes x as input and computes the square of x and adds 2.

The abstraction $\lambda x.x^2 + 2$ does the same thing, except that we do not indicate a name for the function.

The abstraction operator in the term $\lambda x.t$ *binds* x, that is, in t, x is seen as an input. In $\lambda x.x^2 + 2$, the input is x and t is $x^2 + 2$.

Not surprisingly, $\lambda y.y^2 + 2$ denotes the *exact* same function!

Abstractions and α-equivalence

The function $f(x) = x^2 + 2$ takes x as input and computes the square of x and adds 2.

The abstraction $\lambda x.x^2 + 2$ does the same thing, except that we do not indicate a name for the function.

The abstraction operator in the term $\lambda x.t$ *binds* x, that is, in t, x is seen as an input. In $\lambda x.x^2 + 2$, the input is x and t is $x^2 + 2$.

Not surprisingly, $\lambda y.y^2 + 2$ denotes the *exact* same function!

We say that ($\lambda x.x^2 + 2$ and $\lambda y.y^2 + 2$) are α-*equivalent*.

Applications

The construction $t\ s$, where t and s are λ-terms, is called *application*.

It is meant to represent the application of the function t to the function s.

Example: $(\lambda x.y)\ z$ – apply $(\lambda x.y)$ to z

Examples of λ-terms

- x - is a λ-term because it is a variable;

Examples of λ-terms

- x - is a λ-term because it is a variable;
- $\lambda x.x$ - it is an abstraction which represents the identity function;

Examples of λ-terms

- x - is a λ-term because it is a variable;
- $\lambda x.x$ - it is an abstraction which represents the identity function;
- $\lambda x.y$ - a function that always returns y, no matter what is the input x;

Examples of λ-terms

- x - is a λ-term because it is a variable;
- $\lambda x.x$ - it is an abstraction which represents the identity function;
- $\lambda x.y$ - a function that always returns y, no matter what is the input x;
- $\lambda x.(\lambda y.x)$ - a function which takes two arguments and returns the first;

Examples of λ-terms

- x - is a λ-term because it is a variable;
- $\lambda x.x$ - it is an abstraction which represents the identity function;
- $\lambda x.y$ - a function that always returns y, no matter what is the input x;
- $\lambda x.(\lambda y.x)$ - a function which takes two arguments and returns the first;
- $(\lambda x.x)\ y$ - an application of the identity function $(\lambda x.x)$ to an argument y;
  Later we will see that this will evaluate to y, as expected.

Notational conventions

By default, there are no explicit parentheses in λ-terms.

However, we used parentheses in our examples $\big($e.g., $\lambda x.(\lambda y.x)$, $(\lambda x.x)\ y\big)$ to avoid ambiguities.

Notational conventions

By default, there are no explicit parentheses in λ-terms.

However, we used parentheses in our examples $\big($e.g., $\lambda x.(\lambda y.x)$, $(\lambda x.x)\ y\big)$ to avoid ambiguities.

Rule: always use parentheses when needed to avoid ambiguities.

Notational conventions

By default, there are no explicit parentheses in λ-terms.

However, we used parentheses in our examples $\big($e.g., $\lambda x.(\lambda y.x)$, $(\lambda x.x)\ y\big)$ to avoid ambiguities.

Rule: always use parentheses when needed to avoid ambiguities.

Without parentheses it is hard to say whether the λ-term

$$\lambda x.x\ \lambda x.y$$

represents $(\lambda x.x)\ (\lambda x.y)$ or $\lambda x.(x\ (\lambda x.y))$.

Free variables

The set of *free* variables is computed using the function $free : t \to 2^X$ defined below:

- $free(x) = \{x\}$;
- $free(\lambda x.t) = free(t) \setminus \{x\}$;
- $free(t\ s) = free(t) \cup free(s)$.

Free variables example

The free variables of a term are the ones that are not captured by a binder. Example:

$$
\begin{aligned}
\mathit{free}\Big((\lambda x.x)\ (\lambda x.y)\Big) &= \mathit{free}\big((\lambda x.x)\big) \cup \mathit{free}\big((\lambda y.y)\big) \\
&= \Big(\mathit{free}(x) \setminus \{x\}\Big) \cup \Big(\mathit{free}(y) \setminus \{x\}\Big) \\
&= \big(\{x\} \setminus \{x\}\big) \cup \big(\{y\} \setminus \{x\}\big) \\
&= \emptyset \cup \{y\} \\
&= \{y\}.
\end{aligned}
$$

Note that y is not bound, so $\mathit{free}\Big((\lambda x.x)\ (\lambda x.y)\Big) = \{y\}$.

Bound variables

The set of *bound* variables is computed using the $bound : t \to 2^X$ function, defined as

- $bound(x) = \emptyset$;
- $bound(\lambda x.t) = bound(t) \cup \{x\}$;
- $bound(t\ s) = bound(t) \cup bound(s)$.

Bound variables example

The bound variables of a term are the ones that are captured by a binder. Example:

$$
\begin{aligned}
\mathit{bound}\Big((\lambda x.x)\ (\lambda x.y)\Big) &= \mathit{bound}\big((\lambda x.x)\big) \cup \mathit{bound}\big((\lambda x.y)\big) \\
&= \Big(\mathit{bound}(x) \cup \{x\}\Big) \cup \Big(\mathit{bound}(y) \cup \{x\}\Big) \\
&= \big(\emptyset \cup \{x\}\big) \cup \big(\emptyset \cup \{x\}\big) \\
&= \{x\} \cup \{x\} \\
&= \{x\}.
\end{aligned}
$$

Capturing (or non-capture avoiding) substitution

The notation $t[t'/x]$ stands for the term obtained from t by substituting t' for all free occurrences of x.

Formally, $t[t'/x]$ is defined as

1. $x[t'/x] = t'$;
2. $y[t'/x] = y$, if $y \neq x$;
3. $(\lambda x.t)[t'/x] = \lambda x.t$;
4. $(\lambda y.t)[t'/x] = \lambda y.(t[t'/x])$, if $y \neq x$;
5. $(t\ s)[t'/x] = (t[t'/x])\ (s[t'/x])$.

Capturing (or non-capture avoiding) substitution - Example

$$
\begin{aligned}
\big((\lambda x.x)\ (\lambda x.y)\big)[z/y] &= \big((\lambda x.x)[z/y]\big)\ \big((\lambda x.y)[z/y]\big) \\
&= \big(\lambda x.(x[z/y])\big)\ \big(\lambda x.(y[z/y])\big) \\
&= \big(\lambda x.x\big)\ \big(\lambda x.z\big)
\end{aligned}
$$

Capturing (or non-capture avoiding) substitution - Another example

$$(\lambda y.x)[z/y] = \lambda y.(x[y/x]) = \lambda y.y$$

Capturing (or non-capture avoiding) substitution - Another example

$$(\lambda y.x)[z/y] = \lambda y.(x[y/x]) = \lambda y.y$$

Problem: we started with a term which ignores its input, that is $(\lambda y.x)$ and by applying a substitution we obtain a term which does not ignore its input anymore, namely, $\lambda y.y$.

Capturing (or non-capture avoiding) substitution - Another example

$$(\lambda y.x)[z/y] = \lambda y.(x[y/x]) = \lambda y.y$$

Problem: we started with a term which ignores its input, that is $(\lambda y.x)$ and by applying a substitution we obtain a term which does not ignore its input anymore, namely, $\lambda y.y$.

Substitutions are syntactical operations, and they should not change the behaviour of λ-terms.

Capturing (or non-capture avoiding) substitution - Another example

$$(\lambda y.x)[z/y] = \lambda y.(x[y/x]) = \lambda y.y$$

Problem: we started with a term which ignores its input, that is $(\lambda y.x)$ and by applying a substitution we obtain a term which does not ignore its input anymore, namely, $\lambda y.y$.

Substitutions are syntactical operations, and they should not change the behaviour of λ-terms.
A possible fix to this problem is to use an α-equivalent version of $(\lambda y.x)$, say $(\lambda y'.x)$.

$$(\lambda y'.x)[z/y]\lambda y'.(x[y/x]) = \lambda y'.y$$

We obtained a similar λ-term with the one we started with.

Capture-avoiding substitutions

We denote by $t[\![t'/x]\!]$ the substitution that avoids capturing variables:

1. $x[\![t'/x]\!] = t'$;
2. $y[\![t'/x]\!] = y$, if $y \neq x$;
3. $(\lambda x.t)[\![t'/x]\!] = \lambda x.t$;
4. $(\lambda y.t)[\![t'/x]\!] = \lambda y'.\big((t[y'/y])[\![t'/x]\!]\big)$, if $y \neq x$ and y' is a fresh variable;
5. $(t\ s)[\![t'/x]\!] = (t[\![t'/x]\!])\ (s[\![t'/x]\!])$.

Capture-avoiding substitutions - Example

$$
\begin{aligned}
(\lambda y.(x\ y))[\![y/x]\!] &= \lambda y'.\Big(((x\ y)[y'/y])[\![y/x]\!]\Big) \\
&= \lambda y'.\Big(((x[y'/y])\ (y[y'/y]))[\![y/x]\!]\Big) \\
&= \lambda y'.\big((x\ y')[\![y/x]\!]\big) \\
&= \lambda y'.\big((x[\![y/x]\!])\ (y'[\![y/x]\!])\big) \\
&= \lambda y'.(y\ y').
\end{aligned}
$$

Capturing vs. Non-Capturing Substitutions

- Capturing substitution:

$$(\lambda y.x)[y/x] = \lambda y.(x[y/x]) = \lambda y.y;$$

- Capture-avoiding substitution:

$$(\lambda y.x)[\![y/x]\!] = \lambda y'.((x[y'/y])[\![y/x]\!]) = \lambda y'.(x[\![y/x]\!]) = \lambda y'.y.$$

Alpha equivalence

In an abstraction $\lambda x.t$, if the bound variable x is replaced everywhere by a variable y (that does not occur in t) we obtain an α-equivalent form $\lambda y.t[y/x]$ of $\lambda x.t$.

Notation: $t_1 \equiv_\alpha t_2$ denotes the fact that t_1 and t_2 are α-equivalent.

Examples:

1. $\lambda x.x \equiv_\alpha \lambda y.y$;
2. $(\lambda x.x)\ (\lambda y.y) \equiv_\alpha (\lambda z.z)\ (\lambda y.y)$;
3. $(\lambda x.x)\ (\lambda y.y) \equiv_\alpha (\lambda z.z)\ (\lambda z.z)$;
4. $(\lambda x.x)\ (\lambda y.y) \equiv_\alpha (\lambda z.z)\ (\lambda z'.z')$;
5. $(\lambda x.x)\ x \equiv_\alpha (\lambda y.y)\ x$;

Beta-reduction

In λ-calculus there is only one computation rule, called β-reduction:

$$(\lambda x.t)\ t' \rightarrow_\beta t[\![t'/x]\!].$$

The β-reduction rule is the rule that applies an abstraction (a function) to an argument.

The expression $(\lambda x.t)\ t'$ is called *redex* (i.e., the place where the β-reduction happens) and $t[\![t'/x]\!]$ is called *reductum* (i.e., the reduced expression).

Beta-reduction - simple example

$$(\lambda x.x)\ y \rightarrow_\beta x[\![y/x]\!] = y.$$

Here, $(\lambda x.x)\ y$ is the redex, while y is the reductum.

Note that $\rightarrow_\beta$ represents a β-reduction step, while the result of the reduction $x[\![y/x]\!]$ is then computed using the definition of the capture-avoiding substitution.

Another example

$$
\begin{aligned}
\Big(\lambda f.\big(\lambda x.(f\ x)\big)\Big)\ (\lambda x.x)\ z &\to_\beta && \Big(\big(\lambda x.(f\ x)\big)[\![(\lambda x.x)/f]\!]\Big)\ z \\
&= && \Big(\lambda x'.\big(((f\ x)[x'/x])[\![(\lambda x.x)/f]\!]\big)\Big)\ z \\
&= && \Big(\lambda x'.\big(((f[x'/x])\ (x[x'/x]))[\![(\lambda x.x)/f]\!]\big)\Big)\ z \\
&= && \Big(\lambda x'.\big((f\ x')[\![(\lambda x.x)/f]\!]\big)\Big)\ z \\
&= && \big(\lambda x'.(f[\![(\lambda x.x)/f]\!]\ x'[\![(\lambda x.x)/f]\!])\big)\ z \\
&= && \Big(\lambda x'.\big((\lambda x.x)\ x'\big)\Big)\ z \\
&\to_\beta && \big((\lambda x.x)\ x'\big)[\![z/x']\!] \\
&= && \big((\lambda x.x)[\![z/x']\!]\big)\ \big(x'[\![z/x']\!]\big) \\
&= && \big(\lambda x''.(x''[x''/x])[\![z/x']\!]\big)\ z \\
&= && \big(\lambda x''.x''[\![z/x']\!]\big)\ z \\
&= && (\lambda x''.x'')\ z \\
&\to_\beta && x''[\![z/x'']\!] \\
&= && z.
\end{aligned}
$$

The same example, but simplified

$$\begin{aligned}
\Big(\lambda f.\big(\lambda x.(f\ x)\big)\Big)\ (\lambda x.x)\ (\lambda y.y) &\to_\beta \quad \Big(\lambda x'.\big((\lambda x.x)\ x'\big)\Big)\ (\lambda y.y) \\
&\to_\beta \quad (\lambda x''.x'')\ (\lambda y.y) \\
&\to_\beta \quad (\lambda y.y).
\end{aligned}$$

Normal forms

When a λ-term t cannot be reduced (the β-reduction rule cannot be applied) anymore, we say that t is in *normal form*.

Normal forms

When a λ-term t cannot be reduced (the β-reduction rule cannot be applied) anymore, we say that t is in *normal form*.

For instance, $\lambda x.(\lambda y.x)$ is in normal form, while $\lambda x.\big((\lambda y.x)\ z\big)$ is not in normal form, because:

$$\lambda x.\big((\lambda y.x)\ z\big) \to_\beta \lambda x.\big(x[\![z/y]\!]\big) = \lambda x.x.$$

Now, $\lambda x.x$ is a normal form for $\lambda x.\big((\lambda y.x)\ z\big)$.

Normal forms

When a λ-term t cannot be reduced (the β-reduction rule cannot be applied) anymore, we say that t is in *normal form*.

For instance, $\lambda x.(\lambda y.x)$ is in normal form, while $\lambda x.\big((\lambda y.x)\ z\big)$ is not in normal form, because:

$$\lambda x.\big((\lambda y.x)\ z\big) \to_\beta \lambda x.\big(x[\![z/y]\!]\big) = \lambda x.x.$$

Now, $\lambda x.x$ is a normal form for $\lambda x.\big((\lambda y.x)\ z\big)$.

Not all lambda terms have a normal form. For example, the sequence

$$\big(\lambda x.(x\ x)\big)\ \big(\lambda x.(x\ x)\big) \to_\beta \big(\lambda x.(x\ x)\big)\ \big(\lambda x.(x\ x)\big) \to_\beta \cdots$$

is infinite.

Evaluation strategies

Sometimes the β-reduction rule can be applied in several locations under a lambda term.

Example: what is the normal form of $(\lambda x.x)\ \big((\lambda y.y)\ z\big)$?

Evaluation strategies

Sometimes the β-reduction rule can be applied in several locations under a lambda term.

Example: what is the normal form of $(\lambda x.x)\ \big((\lambda y.y)\ z\big)$?
There are two possible ways of finding it:

1. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda y.y)\ z \to z$, or
2. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda x.x)\ z \to z$.

Evaluation strategies

Sometimes the β-reduction rule can be applied in several locations under a lambda term.

Example: what is the normal form of $(\lambda x.x)\ \big((\lambda y.y)\ z\big)$?
There are two possible ways of finding it:

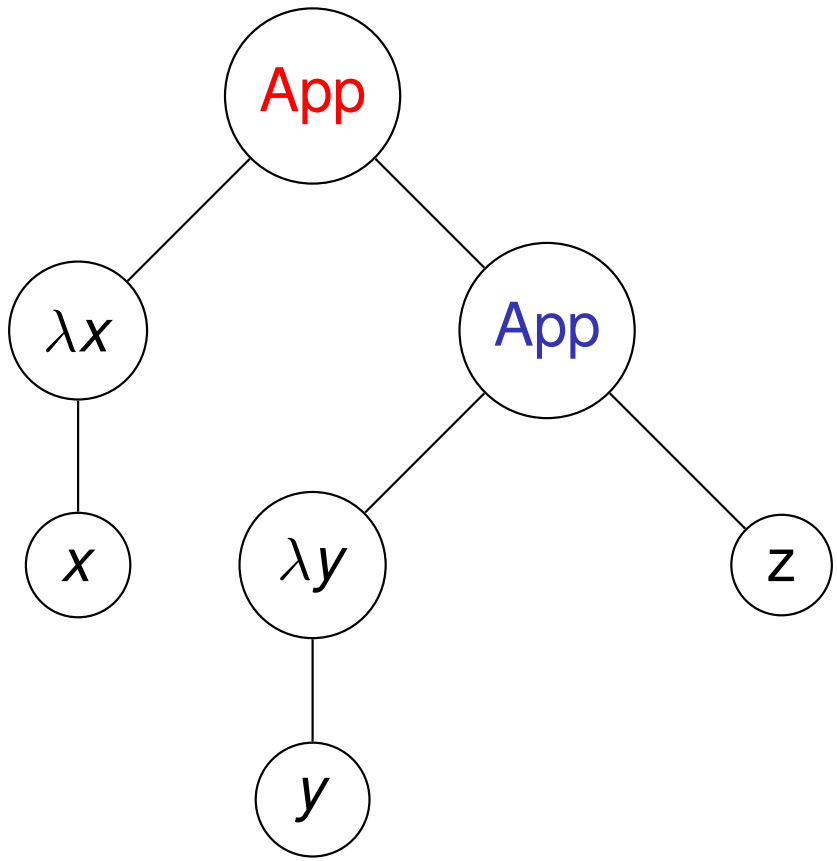
1. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda y.y)\ z \to z$, or
2. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda x.x)\ z \to z$.

Which one should we choose?

Fact: the evaluation strategies that are used to apply the β-reduction rule may impact the performance.

Abstract trees for λ-terms

Abstract tree of $(\lambda x.x)\ ((\lambda y.y)\ z)$:



Full-beta reduction

This strategy is more like a *no strategy* to apply β-reductions. Any of the previous reductions

1. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda y.y)\ z \to z$, or
2. $(\lambda x.x)\ \big((\lambda y.y)\ z\big) \to_\beta (\lambda x.x)\ z \to z$.

are allowed by the full-beta reduction strategy.

Normal order

This strategy says that the leftmost, outermost redex is reduced first.

In our example, there is only one possible reduction:

$$(\lambda x.x)\ \big((\lambda y.y)\ z\big) \rightarrow_\beta (\lambda y.y)\ z \rightarrow z$$

Normal order

This strategy says that the leftmost, outermost redex is reduced first.

In our example, there is only one possible reduction:

$$(\lambda x.x)\ \big((\lambda y.y)\ z\big) \rightarrow_\beta (\lambda y.y)\ z \rightarrow z$$

This strategy allows exactly one successor (up to α-equivalence) for any given lambda term.

Normal order

This strategy says that the leftmost, outermost redex is reduced first.

In our example, there is only one possible reduction:

$$(\lambda x.x)\ \big((\lambda y.y)\ z\big) \rightarrow_\beta (\lambda y.y)\ z \rightarrow z$$

This strategy allows exactly one successor (up to α-equivalence) for any given lambda term.

Also, normal-order is a *non-strict* evaluation strategy because it does not require that the arguments of an abstraction (function) are evaluated before β-reduction.

Call-by-name (CBN)

CBN is a restricted version of normal-order where applications of the β-reduction rule inside an abstraction is forbidden.

Call-by-name (CBN)

CBN is a restricted version of normal-order where applications of the β-reduction rule inside an abstraction is forbidden.

- Call-by-name:

$$(\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big) \to_\beta \lambda x.\big((\lambda y.y)\ x\big) \not\to_\beta .$$

Call-by-name (CBN)

CBN is a restricted version of normal-order where applications of the β-reduction rule inside an abstraction is forbidden.

- Call-by-name:

$$(\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big) \rightarrow_\beta \\ \lambda x.\big((\lambda y.y)\ x\big) \not\rightarrow_\beta .$$

- Normal-order:

$$(\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big) \rightarrow_\beta \\ \lambda x.\big((\lambda y.y)\ x\big) \rightarrow_\beta \\ \lambda x.x.$$

In the latter, the β-reduction inside a $\lambda-$abstraction is allowed.

Call-by-name: advantages

$$(\lambda x.(\lambda y.y))\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) \to_\beta \lambda y.y,$$

CBN is very efficient in this example: $\lambda y.y$ is the only successor here, and there is no need to evaluate the argument $\Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big)$.

Call-by-name: disadvantages

$$
\begin{aligned}
(\lambda x.(x\ x))\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
\Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
\Big(\lambda x.\big((\lambda y.y)\ x\big)\Big)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
(\lambda y.y)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
(\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big) &\to_\beta \\
\lambda x.\big((\lambda y.y)\ x\big) &\not\to_\beta .
\end{aligned}
$$

Because in $\lambda x.(x\ x)$ the input x is used twice, whatever input is passed to this function will be evaluated twice.

Call-by-value (CBV)

CBV: reduce the leftmost, innermost redex which is not inside a lambda abstraction.

The arguments of a function are completely evaluated to *values* first, and then the function is called on those values. This is the most common approach in the mainstream languages.

- Call-by-value example:

$$
\begin{aligned}
(\lambda f.f)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
(\lambda f.f)\ \Big(\lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
\lambda x.\big((\lambda y.y)\ x\big) &\not\to_\beta .
\end{aligned}
$$

CBN vs CBV

- Call-by-name:

$$(\lambda f.f)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) \to_\beta$$
$$(\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big) \to_\beta$$
$$\lambda x.\big((\lambda y.y)\ x\big) \not\to_\beta .$$

Recall that the β-reduction rule inside a lambda abstraction is forbidden and the arguments are not evaluated before applying functions.

- Call-by-value:

$$(\lambda f.f)\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) \to_\beta$$
$$(\lambda f.f)\ \Big(\lambda x.\big((\lambda y.y)\ x\big)\Big) \to_\beta$$
$$\lambda x.\big((\lambda y.y)\ x\big) \not\to_\beta .$$

CBV vs CBN - continued

Sometimes CBV performs a smaller number of steps than CBN:

$$\begin{aligned}
(\lambda x.(x\ x))\ \Big((\lambda z.z)\ \lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
(\lambda x.(x\ x))\ \Big(\lambda x.\big((\lambda y.y)\ x\big)\Big) &\to_\beta \\
\lambda x.\big((\lambda y.y)\ x\big) &\not\to_\beta .
\end{aligned}$$

CBV vs CBN - continued

Sometimes CBV performs a smaller number of steps than CBN:

$$\begin{aligned}(\lambda x.(x\ x))\ \Big((\lambda z.z)\ \lambda x.((\lambda y.y)\ x)\Big) &\to_\beta \\ (\lambda x.(x\ x))\ \Big(\lambda x.((\lambda y.y)\ x)\Big) &\to_\beta \\ \lambda x.((\lambda y.y)\ x) &\not\to_\beta .\end{aligned}$$

However, CBV can yield useless evaluations:

$$\begin{aligned}(\lambda x.(\lambda y.y))\ \Big((\lambda z.z)\ \lambda x.((\lambda y.y)\ x)\Big) &\to_\beta \\ (\lambda x.(\lambda y.y))\ \Big(\lambda x.((\lambda y.y)\ x)\Big) &\to_\beta \\ (\lambda y.y).&\end{aligned}$$

CBN terminates in just a single step on this example!

Functional programming in Haskell

- Integers, Booleans
- Types
- Lists: head, tail, reverse, sum, infinite lists
- Lazy evaluation: examples using functions on infinite lists
- Higher-order functions: map, foldl
- Quicksort

Bibliography/Q&A

Please refer to: Chapter 11:
`https://link.springer.com/content/pdf/10.1007/978-1-84882-914-5.pdf?pdf=button` for other details and interesting features:

1. local environment
2. types
3. pattern matching
4. infinite objects.

Questions?